

Program 7: Polynomial Representation using a Linked List

Aim

To write a C program that represents a polynomial using a linked list, displays it in standard form and evaluates it for a given value of x .

Theory

A polynomial such as $5x^3 - 2x^2 + 4x + 7$ is a sum of terms, each having a **coefficient** and an **exponent**.

It is stored as a linked list with one node per non-zero term: `struct Node { int coef; int exp; struct Node *next; }.`

Nodes are kept in **descending order of exponent**, which makes operations such as addition simple. Terms with the same exponent are combined when inserted.

This representation is efficient for **sparse** polynomials: $x^{1000} + 1$ needs only 2 nodes instead of an array of 1001 entries.

Algorithm

1. Read the number of terms.
2. For each term read coefficient and exponent and insert it in the list in descending order of exponent (add coefficients if the exponent already exists).
3. Display: print each term as $\text{coef } x^{\text{exp}}$ with the correct sign; omit the coefficient 1, print x for exponent 1, and just the number for exponent 0.
4. Evaluate: for each term compute $\text{coef} * x^{\text{exp}}$ and add to the result.

C Source Code

```
/*
 * Program : Polynomial Representation using a Linked List
 *
 * THEORY:
 * - A polynomial such as  $5x^3 - 2x^2 + 4x + 7$  is a sum of terms. Each term
 *   has a COEFFICIENT and an EXPONENT.
 * - A linked list stores it efficiently: one node per non-zero term
 *   struct Node { int coef; int exp; struct Node *next; };
 * - Nodes are kept in DESCENDING order of exponent, which makes later
 *   operations (addition, multiplication) easy.
 * - Terms with the same exponent are combined when inserted.
 * - Sparse polynomials (e.g.  $x^{1000} + 1$ ) use only 2 nodes instead of an
 *   array of 1001 entries.
 */
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coef;
    int exp;
    struct Node *next;
};

/* insert a term keeping the list sorted by descending exponent */
struct Node *insertTerm(struct Node *head, int coef, int exp) {
    if (coef == 0) return head;
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
```

```

n->coef = coef; n->exp = exp; n->next = NULL;

if (head == NULL || exp > head->exp) {          /* new head */
    n->next = head;
    return n;
}
struct Node *t = head;
while (t->next != NULL && t->next->exp > exp) t = t->next;

if (t->exp == exp) { t->coef += coef; free(n); } /* same exp as t */
else if (t->next != NULL && t->next->exp == exp) { /* same exp as next */
    t->next->coef += coef; free(n);
} else {
    n->next = t->next;
    t->next = n;
}
return head;
}

void printPoly(struct Node *head) {
    if (head == NULL) { printf("0\n"); return; }
    for (struct Node *t = head; t != NULL; t = t->next) {
        int c = t->coef;
        if (t == head) { if (c < 0) printf("-"); }
        else printf(c < 0 ? " - " : " + ");
        if (c < 0) c = -c;

        if (t->exp == 0) printf("%d", c);
        else {
            if (c != 1) printf("%d", c);
            printf("x");
            if (t->exp != 1) printf("^%d", t->exp);
        }
    }
    printf("\n");
}

long evaluate(struct Node *head, int x) {
    long result = 0;
    for (struct Node *t = head; t != NULL; t = t->next) {
        long p = 1;
        for (int i = 0; i < t->exp; i++) p *= x;
        result += t->coef * p;
    }
    return result;
}

int main() {
    struct Node *poly = NULL;
    int n, coef, exp, x;

    printf("Enter number of terms: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        printf("Enter coefficient and exponent of term %d: ", i);
        scanf("%d %d", &coef, &exp);
        poly = insertTerm(poly, coef, exp);
    }

    printf("\nPolynomial: ");
    printPoly(poly);

    printf("Enter value of x to evaluate: ");
    scanf("%d", &x);
    printf("P(%d) = %ld\n", x, evaluate(poly, x));

    while (poly != NULL) { struct Node *t = poly; poly = poly->next; free(t); }
    return 0;
}

```

Sample Input

Input: 4 terms entered out of order: (4,1) (5,3) (7,0) (-2,2); evaluate at $x = 2$.

Sample Output (program run with the input above)

```
Enter number of terms: Enter coefficient and exponent of term 1: Enter coefficient and exponent of term
Polynomial:  $5x^3 - 2x^2 + 4x + 7$ 
Enter value of x to evaluate:  $P(2) = 47$ 
```